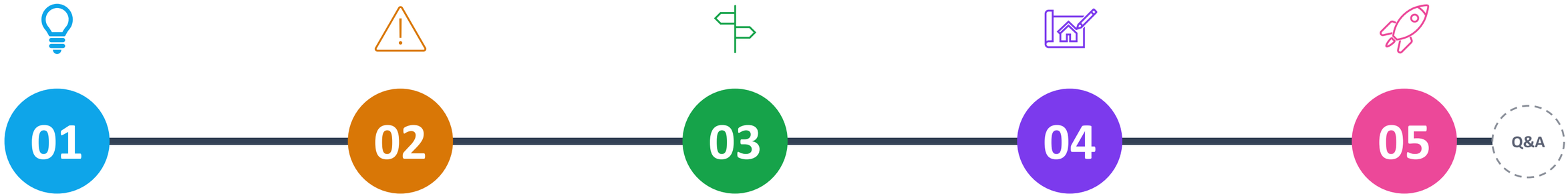
Accidental Data Engineer

A DBA's First Data Pipeline

Tom Huntley | Cloud Rede



Agenda



The Context

How a reporting request became a data project.

The Skill Gap

DBA to Data Engineer. Closer than you think.

The Decisions

Tools, trade-offs and pragmatism.

The Architecture

Pipelines, patterns and building blocks.

The Demo

The real thing, end to end.

An honest look at what it actually takes to build your first data pipeline as a DBA.

Who am I?



Tom Huntley
Senior Cloud Database Administrator (DBA) at CloudRede



SQL Server specialist

HA/DR – performance tuning – query optimisation



Terrible climber

But enjoys it...



Accidental data engineer

You'll see why in a few minutes



The problem

WHAT THEY SAID

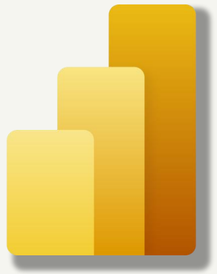
We just need some help building a few reports

- The client, casually

WHAT I ASSUMED

So naturally, I thought...

Power BI dashboards



WHAT WE DID

We flew out to see them.

Sat down with the team. Watched them work. Saw the real pain points first-hand.

The actual problem



Silos, inconsistent data

- Different departments, different systems, zero shared understanding. Finance's number was never Operations' number - nobody could agree whose was right.



Meet Chris...

- A single point of failure...



Fragile

- One formula changes. One column shifts in an export. The whole report breaks and nobody knows why



Every stakeholder was unhappy

- Operations. Finance. Directors. Everyone knew the reporting weas broken. Nobody had time to fix it.

My proposal

Build a reliable, automated analytics platform that replaces manual reporting – and gives every team a single, trusted number

1

Connect

TO SOURCE DATA

Pull directly from the on-premise ERP into a landing zone.

2

Automate

THE DAILY REFRESH

Pipelines run overnight. Data lands before anyone starts work. No more chasing Chris.

3

Transform

WITH BUSINESS RULES

Deduplication, calculations, business logic - all in code, all tested, all version controlled.

4

Deliver

TO POWER BI

Dashboards, one version of the truth.

A slight issue

≠ DATA ENGINEER

(…or does it?)

Upskilling – Getting Data Engineer Ready

1

Courses

Structured learning to build foundational knowledge across the data engineering stack.

PLATFORMS

- Udemy - DP-203 prep, Python for DE
- Microsoft Learn
- Databricks Academy - Spark fundamentals

udemy

Microsoft Learning



2

Sandboxing

Hands-on experimentation in personal lab environments.

TOOLS I BUILT WITH

- Azure Data Factory - pipeline orchestration
- Microsoft Fabric - end-to-end analytics
- Power Automate - workflow automation
- Pentaho & Airbyte - open-source ETL







pentaho

Airbyte

3

Network

Learning from people who've already walked the path – conferences & communities.

KEY CONNECTIONS

- SQLBits / Data Grillen — deep-dive events
- MVP blogs & Twitter/X & LinkedIn
- YouTube / podcasts
- DE content outside the DBA bubble



Tools Evaluated – Extract/Load Choices

	STRENGTHS	CONSIDERATIONS	BEST FOR
Microsoft Fabric UNIFIED ANALYTICS	✓ Power BI deeply integrated – single pane of glass ✓ Unified governance, no separate compute layer	✗ Relatively new – less community documentation ✗ Learning curve for Lakehouse paradigm	*Organisations already invested in the Microsoft ecosystem wanting end-to-end analytics*
Azure Data Factory ORCHESTRATION	✓ Excellent docs, huge community, battle-tested ✓ Mature Copy Activity - just point and pull, no code ✓ Rich monitoring, alerting, and retry logic	✗ Transformation requires external compute ✗ Simple copy jobs still need linked services, datasets, pipelines	*Teams needing mature, production-grade orchestration with deep Azure integration*
Airbyte OPEN-SOURCE INGESTION	✓ 300+ pre-built connectors, vendor-agnostic ✓ Very fast to get started – minutes to first sync ✓ Open-source core – no vendor lock-in	✗ Primarily ingestion – limited orchestration ✗ Self-hosted version requires infrastructure	*Rapid prototyping or multi-source ingestion where speed to first data matters most*

Tools Evaluated – Extract/Load Choices

	STRENGTHS	CONSIDERATIONS	BEST FOR
Microsoft Fabric UNIFIED ANALYTICS	✓ Power BI deeply integrated – single pane of glass ✓ Unified governance, no separate compute layer	✗ Relatively new – less community documentation ✗ Learning curve for Lakehouse paradigm	*Organisations already invested in the Microsoft ecosystem wanting end-to-end analytics*
Azure Data Factory ORCHESTRATION ✓ CHOSEN	✓ Excellent docs, huge community, battle-tested ✓ Mature Copy Activity - just point and pull, no code ✓ Rich monitoring, alerting, and retry logic	✗ Transformation requires external compute ✗ Simple copy jobs still need linked services, datasets, pipelines	*Teams needing mature, production-grade orchestration with deep Azure integration*
Airbyte OPEN-SOURCE INGESTION	✓ 300+ pre-built connectors, vendor-agnostic ✓ Very fast to get started – minutes to first sync ✓ Open-source core – no vendor lock-in	✗ Primarily ingestion – limited orchestration ✗ Self-hosted version requires infrastructure	*Rapid prototyping or multi-source ingestion where speed to first data matters most*

Bronze, Silver, Gold – Transformation Layers

	WHAT	WHY
BRONZE ▪▪▪ *The truth - as it arrived.*	Exact copy from source. Append-only. No transformation whatsoever.	Your safety net. If downstream logic goes wrong, reprocess from Bronze without touching the source again.
SILVER ●●● *The truth - made trustworthy.*	Business rules applied. Deduplication, exclusions, calculated fields.	Clean, trustworthy data. Business logic runs once, centrally - so every downstream consumer gets the same consistent answer.
GOLD ★★★ *The truth - ready to consume.*	Report-ready tables. Data is structured and pre-organised specifically for dashboards and analytics.	Power BI reads from here. Reports load fast because the heavy lifting is already done in the database.

Transformation Decisions

Databricks

SPARK / LAKEHOUSE



- ✓ Industry standard for large-scale data
- ✓ Delta Lake, Spark SQL, notebooks
- ✓ Python + SQL = full flexibility
- ✗ Steep learning curve for Spark concepts
- ✗ Overkill for small-medium data volumes

ADF Dataflows

LOW-CODE / VISUAL ETL



- ✓ Visual drag-and-drop – no code needed
- ✓ Already inside ADF – no new tooling
- ✗ Limited flexibility for complex logic
- ✗ Uncertain future – Fabric Data Flows Gen2

Stored Procedures

T-SQL / AZURE SQL



- ✓ Pure T-SQL – team already skilled
- ✓ Zero new tooling – shipped faster
- ✗ Scalability ceiling on very large data

Transformation Decisions

Databricks

SPARK / LAKEHOUSE



✓ Industry standard for large-scale data

✓ Delta Lake, Spark SQL, notebooks

✓ Python + SQL = full flexibility

✗ Steep learning curve for Spark concepts

✗ Overkill for small-medium data volumes

ADF Dataflows

LOW-CODE / VISUAL ETL



✓ Visual drag-and-drop – no code needed

✓ Already inside ADF – no new tooling

✗ Limited flexibility for complex logic

✗ Uncertain future – Fabric Data Flows Gen2

Stored Procedures

T-SQL / AZURE SQL



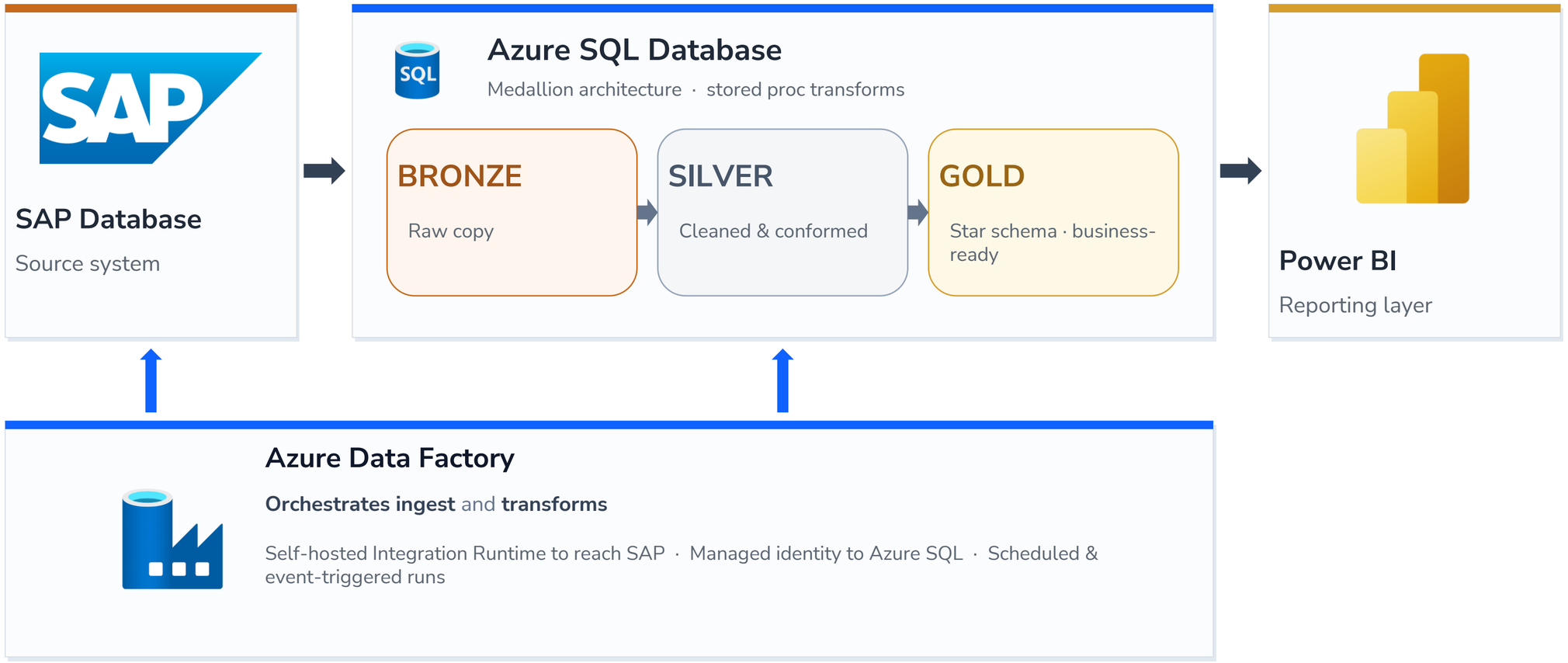
✓ Pure T-SQL – team already skilled

✓ Zero new tooling – shipped faster

✗ Scalability ceiling on very large data

✓ CHOSEN

Architecture Snapshot



ADF – The Building Blocks

INFRASTRUCTURE Shared resources the pipeline uses

4 Linked Services

THE CONNECTIONS

Connection strings to your data sources and destinations: SQL Server, Blob Storage, APIs.

Think: SSMS connection / Linked Server

5 Datasets

THE DATA SHAPE

Points to the specific table, file, or folder. Defines the schema of what you're moving.

Think: Table or View in a query

6 Integration Runtime

THE EXECUTION ENGINE

Where the work actually runs. Azure-hosted for cloud, or self-hosted to reach on-prem SQL Server.

Think: The server your SSIS runs on

ORCHESTRATION uses INFRASTRUCTURE

ORCHESTRATION The flow: what runs, in what order

1 Trigger

WHAT STARTS IT

Kicks off your pipeline. Can be a schedule, an event (file arrives), or a manual run.

Think: SQL Agent Job schedule

3 Activities

THE ACTUAL WORK

Individual tasks: Copy Data, Stored Procedure, Lookup, ForEach loop, and more.

2 Pipeline

THE CONTAINER

The wrapper that holds all your activities in order. Handles flow, dependencies, and errors.

Think: SSIS Package

Making pipelines smart with metadata

THE PATTERN

1 Lookup
Read control table

→

2 ForEach
Loop active rows

→

3 Copy Data
Source → Sink

→

4 Stored Procs
Transform layers

Control Table THE BRAIN

A table in your database that tells the pipeline what to move. Each row = one table to copy.

Columns:

`source_schema` • `source_table` • `sink_schema` • `sink_table` • `load_order` • `is_active`

Add or remove tables without ever editing the pipeline.

Parameterisation THE FLEXIBILITY

Nothing is hardcoded. Datasets and activities read values at runtime using expressions.

Dataset parameters:

```
@dataset().schema_name
@dataset().table_name
```

One dataset, one pipeline - handles every table/source.

One pipeline • Any number of tables • Zero code changes



It’s demo time!

Key Takeaways

We’re engineers - we have tools, not rules.

There is no single right answer. Pick what works for your team, your data, and your constraints — then own it.

The tech matters - but it’s not the whole picture.

Knowing your tools is only half the job. Navigating stakeholders, managing expectations, and earning trust across the business is what turns a pipeline into a platform people actually use.

Start before you’re ready.

The biggest risk isn’t choosing the wrong tool — it’s never starting. Build the first pipeline, learn, iterate.



Questions?

Resources

Feedback

